CIPHER: A Collaborative IDS Placement and Scheduling Framework for Host-Oriented Mimicry Attack Detection in IoT Networks

Surja Sanyal, Manas Khatua, *Member, IEEE*, and Pratik Chattopadhyay, *Member, IEEE*

***Abstract*—The Internet of Things (IoT) enforces security measures, in terms of intrusion detection systems (IDSs), to defend against attackers. Host-oriented mimicry attacks (HMAs) imitate network and host behaviors to evade detection by IDSs. Collaborative IDSs (CIDSs) in IoT networks improve their HMA detection prowess through consultation with the peer IDSs of collaborating devices. However, the existing CIDSs did not deploy placement and scheduling algorithms to ensure that collaborators can readily respond with their IDS prediction to peer requests. Such an always-on approach for all collaborators consumes high network resources and impacts total network runtime. Therefore, this work proposes a collaborative IDS placement and scheduling framework (`CIPHER`) for IoT networks. `CIPHER` uses a hybrid CIDS for HMA detection with high accuracy, a communication range-based placement scheme to ensure seamless network coverage, a behavior monitor-based lightweight scheduling scheme to ensure resource-saving, and a device state history-based objective selection scheme to ensure both high HMA detection rates during frequent attacks and resource-saving during benign runs. Extensive testbed experimentation has shown that `CIPHER` achieves 87.13% HMA detection accuracy while maintaining comparable collaboration and execution overheads, and demonstrating 39.62% fewer IDS executions than the state-of-the-art under frequent attack conditions. Additionally, testbed experimentation also confirms that `CIPHER` saves 78.46% IDS executions during scarce attack conditions with 99.83% HMA detection accuracy while maintaining comparable collaboration and execution overheads to that of the state-of-the-art.**

***Index Terms*—Collaborative intrusion detection system (CIDS), Internet of Things (IoT), intrusion detection system (IDS) placement and scheduling, mimicry attack.**

NOMENCLATURE

t	Deployment cycle.
V	Network devices.
N	Network behavior.
S	Collaboration request.
E_{th}	Device state threshold.
G	Communication graph.
L	Communication edges.
H	Host device behavior.
C	Collaboration response.
P	Placement device list.
n	Total number of features in network and device behaviors.
A_C	Minimum attacks required for peer device collaboration.
A_{th}	Attack tolerance threshold to raise an attack alert.
D	Decision to execute IDS in the current cycle t.
F_{th}	Monitor AE reconstruction error threshold to execute IDS.
S_{th}	Peer request threshold to execute IDS.
C_{th}	Peer response threshold to raise an attack alert.

Received 18 July 2025; revised 23 February 2026; accepted 25 April 2026. Date of publication 30 April 2026; date of current version 9 July 2026. *(Corresponding author: Surja Sanyal.)*

Surja Sanyal is with the Department of Computer Science and Engineering, IIT Guwahati, Guwahati, Assam 781039, India, and also with IIT (BHU) Varanasi, Varanasi, Uttar Pradesh 221005, India (e-mail: surja.sanyal@iitg.ac.in).

Manas Khatua is with the Department of Computer Science and Engineering, IIT Guwahati, Guwahati, Assam 781039, India (e-mail: manaskhatua@iitg.ac.in).

Pratik Chattopadhyay is with the Department of Computer Science and Engineering, IIT (BHU) Varanasi, Varanasi, Uttar Pradesh 221005, India (e-mail: pratik.cse@iitbhu.ac.in).

Digital Object Identifier 10.1109/JIOT.2026.3689273

I. INTRODUCTION

THE requirement for intrusion detection systems (IDSs) [1] in the Internet of Things (IoT) networks is imperative to defend against attacks [2]. Host-oriented mimicry attacks (HMAs) imitate benign host and network behaviors to evade detection [3], [4]. Positional hybrid IDSs [5] combine network-based (NIDS) [6] and host-based (HIDS) [7], [8] IDSs to detect network and host attack footprints [9], [10]. Hybrid IDSs combine signature or rule-based IDSs [11], and anomaly-based IDSs [12] to detect known attack footprints and unknown device or network behavior patterns [13]. Anomaly-based IDSs often reinforce their performances using machine learning (ML) or deep learning (DL) based tools [14], [15] to help detect multistep, stealthy HMA attacks [16]. Thus, hybrid IDSs evaluate host behavior using network context to detect HMAs with greater accuracy [9], [17]. However, IDSs consume network resources during each execution and require placement and scheduling algorithms to make their implementations resource-aware. Intra-IDS communication is reduced by placing both the network traffic and the host behavior-based implementations in the host device, albeit with high device resource consumption [9]. Centralized IDSs place their network component in the router to reduce device resource consumption [10], [18]. However, this strategy has a very high communication overhead. Random-based placement schemes are used to reduce device resource consumption while avoiding being modeled by HMA-like advanced attacks [19]. However, a standalone implementation of this strategy suffers from a low HMA detection rate. Collaborative IDSs (CIDSs) in IoT networks improve their attack detection prowess further by exchanging attack-detection information between peer devices [20], [21], [22]. Three collaboration strategies, namely, centralized [23], hierarchical [24], and distributed strategies [25], [26], are followed by CIDSs. Due to its lower communication overhead, the distributed strategy is most suitable for IoT networks. However, this approach exhibits the lowest HMA detection accuracy due to its partial view of the network.

To ensure a successful collaboration, peers have to constantly execute their IDSs and be ready with their responses at all times, leading to heavy resource consumption. Random-based peer selection [26] and traffic load-based scheduling schemes are used to mitigate resource consumption [7]. However, the former strategy fails for sparse networks with fewer collaborators, and the latter strategy cannot ensure collaboration-readiness due to the scheduling scheme. Thus, placement and scheduling schemes for CIDSs remain underexplored. Therefore, this work proposes a collaborative IDS Placement and scheduling framework (`CIPHER`) for IoT networks. The major contributions of this work are as follows.

1) A collaborative hybrid IDS scheme (`COLLAB`) to detect HMA attacks using peer IDS collaboration.
2) A communication range-based IDS placement scheme (`SELECT`) to select suitable devices for IDS execution.
3) A request-response and behavior-monitor based scheduling scheme (`EXECUTE`) to decide when the selected devices run which component of the IDS framework.
4) A device state history-based primary objective switching scheme (`SWITCH`) to select the emphasis distribution between HMA detection accuracy and resource saving.
5) An experimental evaluation to assess the performance of the proposed framework `CIPHER`, which comprises the `COLLAB`, `SELECT`, `EXECUTE`, and `SWITCH` schemes.

The remainder of this article is organized as follows. Existing works in CIDS placement and scheduling are summarized in Section II. The proposed mechanism is presented in Section III. The evaluation results are presented in Section IV. Finally, the conclusion is presented in Section V.

II. Related Work

HMAs are advanced attacks that imitate benign host and network behaviors to evade detection by IDSs while carrying out malicious activities from inside the affected host device [3], [4]. IDSs use network traffic [6] or host behavior [7], [8] for attack detection. Positional hybrid IDSs [5] analyze both network traffic and host behavior for attack detection [9], [10]. IDSs can detect attacks based on known attack signatures or rules [11], or find previously unseen behavior patterns or anomalies [12]. ML/DL based tools are often utilized with IDSs that detect anomalies, due to the difficulty in manually tracking the large amount of data generated by IoT networks [14], [15]. It also helps detect stealthy attacks like HMAs that often carry out their activities in multiple steps interlaced with legitimate host behavior sequences [16]. Hybrid IDSs combine signature, rule, and anomaly-based attack detection approaches for better HMA detection accuracy [9], [13], [17]. However, their device implementations consume host resources each time they execute [10], mandating the use of IDS placement and scheduling schemes [7]. Communication between the host and network implementations of IDSs is reduced by placing both implementations in the host device [9]. However, this consumes more device resources than executing only the HIDS implementation. Placing the network implementation on the router reduces the resource demand on the devices [10], [18]. However, the router needs to synchronize frequently with the devices, increasing the communication overhead. Random selection of a few devices among the available network devices to place the IDS is another approach to reduce overall network resource consumption [19]. However, a random-based placement strategy suffers from low HMA detection accuracy.

To increase the HMA detection accuracy, CIDSs collaborate with peers by exchanging information related to attack detection [20], [21], [22]. Centralized collaboration strategies have a wholesome view of the network and provide the best HMA detection accuracy [23]. Hierarchical collaboration strategies [24] compromise on the HMA detection accuracy to reduce the high communication overhead of the centralized strategies. Distributed collaboration strategies [25], [26] offer the lowest communication overhead and are the best for resource-constrained IoT networks. However, they also have the lowest HMA detection accuracy due to a local view of the network. Random-based peer selection from the available peers also reduces the communication overhead [26]. However, this only works with dense networks where there are several collaborators available. CIDS-based peers execute their IDSs constantly to provide ready responses to collaboration requests. This continuous IDS execution leads to heavy resource consumption. IDS scheduling schemes, based on network traffic load, reduce the resource demands on the host devices [7]. However, they interfere with collaboration schemes as devices do not have their responses ready due to the enforcement of the scheduling scheme. Therefore, the selection of devices to host the IDS (placement) and the decision of when to execute the IDS (scheduling) in CIDS environments, such that they can maintain a high HMA detection accuracy while saving device resources, remain underexplored. Finally, the summary of the notable features of the existing works is presented in Table I.

III. Proposed Methodology

At the expiry of each predefined time period t (henceforth, cycle), the proposed framework `CIPHER` executes to detect attacks on the network. `CIPHER` consists of four modules. The *detector module* detects HMAs (discussed in Section III-B). The *placement module* decides which devices will run the IDS (discussed in Section III-C). The *scheduling module* decides when to execute the IDS (discussed in Section III-D). The *objective module* decides the emphasis distribution of the framework between resource saving and attack detection (discussed in Section III-E). The attack model used in this work is presented in Section III-A. The frequently used symbols and notations in this work are presented in Nomenclature.

TABLE I
CIDS PLACEMENT AND SCHEDULING FOR HMA DETECTION IN IoT

Framework	HDA	RC	RA	CO	IC	CP	CS
Hawkware [9]	High	High	×	NP	×	×	×
Panop [10]	Med	Med	×	Med	×	×	×
WindRain [19]	Low	Low	×	NP	×	×	×
CADeSH [23]	NV	Low	×	Low	✓	×	×
DIDMA [24]	NV	Low	×	Low	✓	×	×
ECCID-SC [25]	Med	High	×	High	✓	×	×
Whirlpool [26]	Low	Med	×	NP	✓	×	×
CONTRAST [7]	Low	Low	✓	NP	×	×	×
This work	High	Med	✓	Med	✓	✓	✓

HDA = HMA Detection Accuracy, RC = Resource Consumption, RA = Resource Awareness, CO = Communication Overhead, IC = IDS Collaboration, CP = Collaborative Placement, CS = Collaborative Scheduling, Med = Medium, NV = Not Available, NP = Not Applicable.

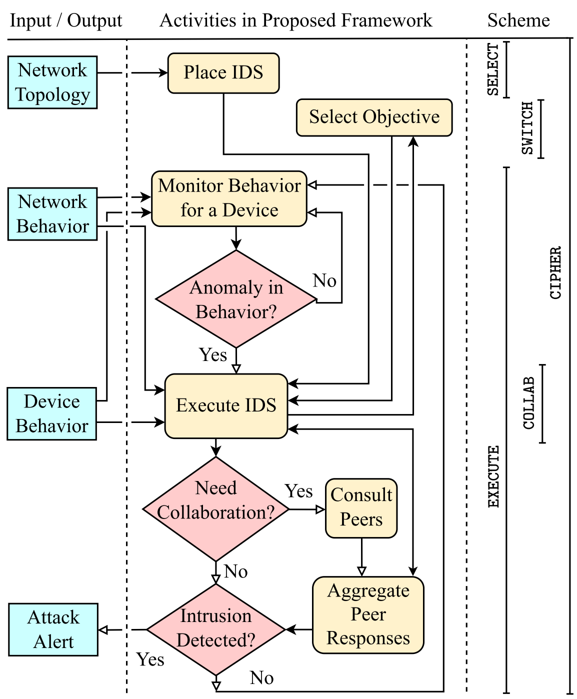


Fig. 1. Flowchart of the activities of the proposed CIDS framework `CIPHER`.

Fig. 1 presents the flowchart of the activities in the proposed framework `CIPHER`. The network topology is used to select the devices for IDS placement. The network and the host behaviors together are monitored to decide on IDS execution. The execution decision also depends on the objective of the framework, decided based on the past device states (*attack* or *benign*). This objective decides the priority of the framework between high HMA detection accuracy and resource saving. If the outcome of the IDS execution is conclusive, then an attack alert is raised. Otherwise, peers are consulted, and their responses are aggregated to arrive at a conclusive decision. Fig. 2 presents the modular implementation of `CIPHER` in a device. Initially, the IDS is placed on the selected host devices. Thereafter, the network and host behaviors of these devices are monitored by the scheduling module. This module decides to execute the IDS at the expiry of the current deployment time period (i.e., cycle). The execution of the IDS is also decided by the framework objective based on the device state history. When scheduled, the detector module executes the IDS. If the outcome of the IDS execution is inconclusive, then peer

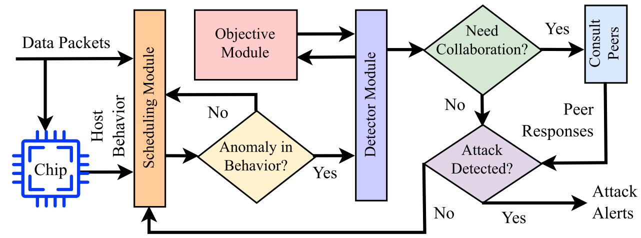


Fig. 2. Device implementation of the proposed CIDS framework `CIPHER`.

IDSs are consulted for their decisions. Post aggregation of the collaboration responses, a decision is made on the device state.

A. The Host-Oriented Mimicry Attack (HMA) Model

HMAs mimic legitimate host and network behaviors to evade detection by IDSs while they carry on malicious activities stealthily from inside a host device [3], [4]. HMAs are advanced attacks and evolve constantly, with several implementations existing in the literature. Using openly available detection algorithms, such attacks improve on their capabilities to evade IDS detection. One of the approaches for designing HMA attacks is by inserting small pieces of attack code at randomly chosen positions in the legitimate host device process execution sequence [27]. The following HMAs were implemented for this work using the aforementioned approach.

1) IoT networks rely on pseudorandom processes, such as the time slotted channel hopping (TSCH) scheme, to gain immunity against attackers [28]. The HMA designed for this work modifies such random behavior of the device, such as ad hoc sensor reading communications, to improve its probability of evading detection. The attack is aimed at faster device resource depletion, leading to the gradual unavailability of the affected devices.
2) Spywares, adapted to mimic legitimate behavior to evade IDSs, can monitor, collect, and communicate user data [29], [30]. Consider an attack in which sensor data is saved to a file. System calls are required for file operations. When observed, they appear in ordered subsequences (complex temporal dependencies) like [31]

$$\ldots, \textit{syscall}\_1(), \ldots, \textit{syscall}\_2()$$
$$, \ldots, \textit{syscall}\_3(), \ldots.$$

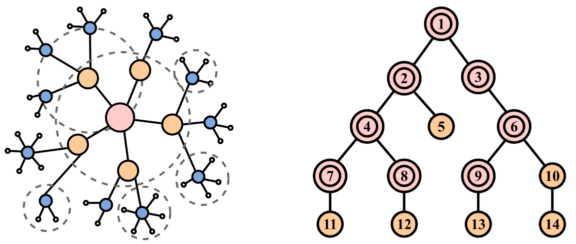
To evade system call detectors, a two-stage attack strategy is employed. First, system calls for file-writing are replaced with false data injection to evade detection and adapt the IDS to malicious data. IDSs often relax behavioral monitoring during long, perceived-benign sequences to conserve energy. In the second stage, sparse file-writing operations are inserted at incremental frequencies to leverage this relaxed monitoring state for IDS evasion.

B. The Detector Module (`COLLAB`)

The evaluation of the network and host behaviors of a device together provides an advantage in the detection of HMAs [9]. Therefore, a hybrid IDS that combines NIDS and HIDS capabilities is an obvious choice for the detector module.

TABLE II
FEATURE VARIABLES TAKEN AND PREPROCESSING APPLIED

Behavior	Feature name	Preprocessing applied
Device	CPU energy consumed	None
	Tx energy consumed	None
Network	Packet size	None
	Timestamp	Consecutive packet difference
	Source IP	Ordinal encoded format [35]
	Destination IP	Ordinal encoded format
	Protocol	One-hot encoded format [35]
	Source port	One-hot encoded format
	Destination port	One-hot encoded format



(a) (b)

Fig. 3. Placement scheme of CIPHER. (a) Multihop network with underlying communication ranges. (b) Nodes selected for IDS placement.

Additionally, since behavior data is a temporal sequence, long short-term memory (LSTM)-based [32] detectors have been chosen for analyzing the same. Furthermore, the autoencoder (AE) [33] architecture has been adopted for the IDS as it excels in unsupervised training. However, since LSTM layers are resource-consuming, only the encoder of the AE is equipped with the same. For fast reconstruction of the encoded features, a fully connected (FC) [34] layer is adopted for the decoder of the AE. Lastly, the network and the host behavior features are clustered based on correlation and mapped to an ensemble, since splitting one neural network (NN) into an ensemble with the same total number of features reduces the number of trainable parameters, making lightweight detectors [10].

The shallow AE-based architecture of the detector module is equipped with LSTM-based ensemble encoders for network and host behavior encoding, and one FC-based decoder for fast feature reconstruction [18]. These AEs are trained using the benign network and host behaviors. The trained AEs are used as the attack detectors for each deployment cycle, if executed. The features used for network and host behavior analysis are presented in Table II. On detecting A_{th} attack attempts, the IDS raises an attack alert without any requirement for collaboration. Otherwise, if the detected attack attempts are less than A_{th}, but greater than A_C, then collaboration is sought from peer IDSs. If both the above conditions are not met, then the device state is declared as benign for the current cycle t.

C. The Placement Module (`Select`*)*

Fig. 3(a) presents a multihop network representing an IoT environment. Each device has multiple neighbors to collaborate with. Note that the presented destination-oriented directed acyclic graph (DODAG) is not the only possible DODAG for the underlying communication range-based graph $G = (V, L)$. Thus, it is possible that a device (interchangeably, a node or mote) changes its parent during deployment, altering the DODAG. A selected parent for IDS placement may lose its

Algorithm 1 Proposed IDS Framework `CIPHER`

Input: $G = (V, L), N_t, H_t$
Output: D_t^V the current cycle IDS execution decision
1 **Function** `SELECT`$(G = (V, L))$**:**
2 $P \leftarrow \phi$ initialize placement vertex list
3 $C_{th}, E_{th}, S_{th} \leftarrow 0.5$ initialize thresholds
4 $V_S \leftarrow$ `sort`(V) in descending neighbor count $|L|$
5 **for each** *vertex* $v \in V_S$ **do**
6 **if** *v is an internal node* **then** $P \leftarrow P \cup \{v\}$
7 **for each** *deployment cycle* t **do**
8 $P_t \leftarrow$ get $[\ |V|/2, |P|\ ]$ random vertices from P
9 **for each** *vertex* $v \in V$ **do**
10 **if** $v \in P_t$ **then** `EXECUTE`$(N_t, H_t, C_{th}, S_{th})$
11 **else** `SWITCH`(E_{th})
12 **Function** `EXECUTE`$(N_t, H_t, C_{th}, S_{th})$**:**
13 $T_t \leftarrow N_t || H_t$
14 $M_t \leftarrow$ get decision of monitor AE using T_t, F_{th}
15 $S_t \leftarrow$ **count** collaboration requests
16 $C_t \leftarrow$ **mean** of collaboration responses
17 **if** $M_t = 1$ **or** $S_t > S_{th}$ **or** $C_t > C_{th}$ **then**
18 $D_t^p \leftarrow$ execute IDS for cycle t `// COLLAB`
19 $S_{th} \leftarrow S_{th} - (S_t - S_{t-1})/2$ `// Update` S_{th}
20 $C_{th} \leftarrow C_{th} - (C_t - C_{t-1})/2$ `// Update` C_{th}
21 **else**
22 $D_t^p \leftarrow$ do not execute IDS for cycle t
23 $S_{th} \leftarrow S_{th} + (S_t - S_{t-1})/2$ `// Update` S_{th}
24 $C_{th} \leftarrow C_{th} + (C_t - C_{t-1})/2$ `// Update` C_{th}
25 **Function** `SWITCH`(E_{th})**:**
26 $E_t \leftarrow$ get EWMA of last $t-1$ device states
27 **if** $E_t > E_{th}$ **then**
28 $D_t^p \leftarrow$ execute IDS for cycle t `// COLLAB`
29 $E_{th} \leftarrow E_{th} - (E_t - E_{t-1})/2$ `// Update` E_{th}
30 **else**
31 $D_t^p \leftarrow$ do not execute IDS for cycle t
32 $E_{th} \leftarrow E_{th} + (E_t - E_{t-1})/2$ `// Update` E_{th}

children between deployment phase cycles. However, children often switch between parents with more neighbors, resulting in seamless IDS coverage. Fig. 3(b) presents an example DODAG to demonstrate the same. Leaf nodes are not considered for placement as they lead to lower network coverage. Therefore, the internal nodes P with more neighbors in their communication range (here, nodes 1 to 4 and 6 to 10) are suitable for IDS placement. As the root node connects with multiple subtrees, it is always chosen for placement. To prevent modeling by adversaries [19], a random subset of the internal nodes P in the range $[\ |V|/2, |P|\ ]$ is chosen for IDS placement at each cycle t. The upper limit ensures minimal motes for IDS placement with maximum coverage. The lower limit ensures low resource overhead while providing 2-hop coverage for leaf nodes. The DODAG presented in Fig. 3(b) has $|P| = 9$ and $|V|/2 = 7$. Therefore, for a cycle t, the IDS is placed in a random set of nodes $P_t \subseteq P$ with $|P_t| \in [7, 9]$ (here, $|P_t| = 8$), marked by double circles. Lines 1–11 of Algorithm 1 present the IDS placement process.

D. The Scheduling Module (`Execute`*)*

Note that the list of placement devices per cycle is long, as it ranges in $[\ |V|/2, |P|\ ]$. This is counterintuitive to the motivation of resource-saving. To save resources, the framework is scheduled such that it executes only when required. Feature-monitoring and request-response-based schemes are used to

TABLE III
EXPERIMENT PARAMETERS FOR SIMULATION [36] AND TESTBED [37]

Parameter	Value	Parameter	Value
Radio	UDGM: Distance loss	Topology	Random / Grid
Routing	RPL / RPL lite	Tx range	50 m radius
Platform	Z1 / M3 / A8 / RPi3	Area covered	200×200 m^2
Total motes	10/20/30/40/50/60	Attack motes	10% of total
Total time	10 hours	Attack time	Last 8 hours

achieve the same. These schemes are discussed in detail next. Lines 12–22 of Algorithm 1 present the overall scheduling scheme applied to each device selected for IDS placement. The network behavior (N_t) and the host behavior (H_t) for the cycle t are its inputs. The collaboration request and response thresholds for IDS execution, S_{th} and C_{th}, respectively, are updated as half of the change in the cycle (shown in lines 19, 20, 23, and 24) to dampen sudden spikes in mote behavior. A high update rate makes the collaboration rare, while a low value exhausts device resources. The final execution decision D_t^V for all vertices V in cycle t is the output of the algorithm.

1) The Feature Monitoring Scheme: As the number of features increases, it becomes more difficult to monitor them for any stealthy attack. Therefore, ML-based tools are best suited for the same. An FC-based shallow AE with one input, one output, and one bottleneck layer is used for feature monitoring. The dimensions of the input and output layers are equal to the total number of network and host features n. The bottleneck layer has a dimension of $\lfloor\sqrt{n}\rfloor$. This AE is both lightweight and fast, eliminating any need for human intervention. The concatenated network and host features are fed to this AE. The reconstruction error is compared against the maximum value during training F_{th} to make the decision D_t^P for IDS execution, as presented in line 14 of Algorithm 1.

2) The Request-Response Scheme: The request-response scheme can also trigger an IDS execution. If a host receives at least S_{th}% collaboration requests in a cycle, it ignores scheduling and executes the IDS. If a device receives C_{th}% peer responses, it ignores scheduling and executes the IDS. After the collaboration, it then reverts to the resource-saving mode. This enables the IDS framework to emphasize resource-saving. HMA detection is prioritized when directed by the monitoring or the objective modules. Lines 15 and 16 of Algorithm 1 present the request-response scheme.

E. Objective Module (`SWITCH`)

Different device states mandate different objectives for IoT networks. During long, benign runs of the device deployment phase, the primary focus is to save device resources. However, the aim of device resource conservation must be overridden in favor of the highest achievable HMA detection accuracy during frequent attacks. An exponential weighted moving average (EWMA) over the device state history achieves the same. To maintain records of attacks, recent states are assigned reduced weight (α). Very low values of α reduce stealth attack detection capabilities, while very high values reduce its ability to adapt to the current device state and lead to high resource consumption. At the expiry of each cycle t, this EWMA is computed using (1) on the previous $t-1$ results

$$E_t = \alpha \times E_{t-1} + (1-\alpha) \times E_{t-2}. \tag{1}$$

This is compared against an adaptive threshold E_{th}, updated at each cycle (shown in lines 29 and 32). A high threshold updation rate depletes resources, while a low one encourages sustained attack states. On violation, the IDS is mandatorily executed for cycle t, overriding the scheduling scheme. Lines 25–31 of Algorithm 1 present this module.

IV. PERFORMANCE ANALYSIS AND DISCUSSION

Here, the theoretical analysis, the experimental setup, and the evaluation of the proposed framework have been discussed.

A. Accuracy and Communication Tradeoff Analysis

The collaboration cost associated with a node i having j neighbors in its collaboration radius is given by

$$cost_i^j = query + response \le 2 \times j = 2j. \tag{2}$$

Since there can be a maximum of P nodes running the IDS, the overall collaboration cost of the network with V nodes is

$$cost_P \le \sum_{i=1}^{P} cost_i^j = P \times 2j = 2Pj \le 2PV \approx O\left(V^2\right). \tag{3}$$

Thus, the collaboration cost is capped at $2PV$ for each node collaborating with the whole network. However, the collaboration radius is a small integer with fewer collaborating peers. Furthermore, the number of nodes running the IDS at each cycle is a subset of P. Thus, in reality, the collaboration cost is $cost_P = O(P \times j) \approx O(V)$ as j is a small integer. The accuracy from collaboration of a model depends on j as

$$Acc_P \le \sum_{i=1}^{P} \frac{\sum_j D^j}{j} = P \times \frac{\sum_j D^j}{j} \le \frac{P^2}{j} \approx O\left(V^2\right). \tag{4}$$

Therefore, a larger number of collaborators ensures that a majority of the neighborhood detects the attack, thereby increasing the framework's accuracy. Thus, accuracy increases with collaborators, but so does the communication overhead.

B. Experiment Environment Setup

The experiment setup, system implementation, state-of-the-art frameworks, and the comparison metrics are discussed here.

1) Dataset and Testbed: The Cooja simulator in Contiki-NG [36] and the FIT IoT-LAB testbed [37] were used for data collection. The parameters used in the experiments are presented in Table III. The motes were programmed to send sensor readings to the root using a multihop topology. Host and network behaviors were captured using the Energest module of Contiki-NG, the built-in packet analyzer in Cooja, and the IoT-LAB monitoring profiles. HMA attacks were implemented by altering device firmware before flashing it onto the hardware.

2) System Implementation: `CIPHER` was implemented using Python scripts, and its IDSs were trained using two Intel Xeon Gold 6230 CPUs running at 2.10 GHz, having a total of 80 logical cores and 512 GB of memory for 250 epochs with a batch size of 32, a learning rate of 0.05, and timesteps of 16. Cycles of duration $t = 200$ s were used. The EWMA weight for recent device states was set to $\alpha = 0.7$. Attack tolerance was set to $A_{th} = 50$ to encourage collaboration by all the frameworks within a two-hop neighborhood.

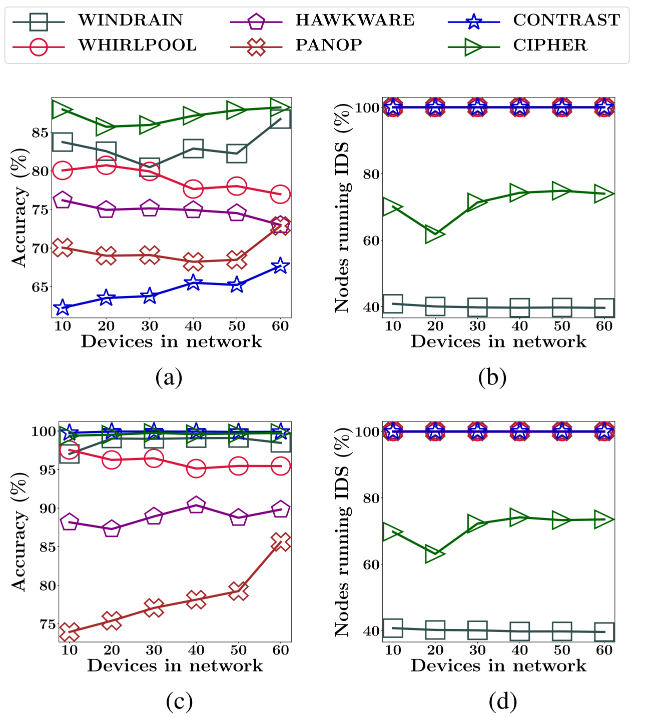


Fig. 4. HMA detection accuracy and IDS placement of all the frameworks during frequent and scarce attack conditions for varying network sizes (a) Under frequent attack. (b) Under frequent attack. (c) Under scarce attack. (d) Under scarce attack.

3) State-of-the-Art Frameworks: Five state-of-the-art frameworks were compared, namely, *WINDRAIN* [19] that uses random IDS placement, *WHIRLPOOL* [26] that chooses random collaborators, *HAWKWARE* [9] that uses distributed detection, *PANOP* [10] that uses centralized ensemble IDSs, and *CONTRAST* [7] that uses IDS scheduling, with CIPHER. Peer IDS collaboration was enabled for all frameworks.

4) Evaluation Metrics: The frameworks were evaluated on accuracy (ACC), precision (PRC), recall (REC), and $F1$-scores (F1S). Additionally, true positive rate (TPR), false positive rate (FPR), true negative rate (TNR), and false negative rate (FNR) were also compared. Furthermore, HMA detection accuracy, peer IDS communication overhead, IDS execution overhead, and energy savings due to the placement and scheduling of the frameworks were also compared. The results are presented in Tables IV and V, and Figs. 4–7.

C. Experiment Results and Analysis

Table IV presents the overall performance of the frameworks during frequent and scarce attack conditions. The best values per metric are highlighted using a bold font. During frequent attacks, it is observed that CIPHER portrayed the highest accuracy, precision, recall, and F1-score among the frameworks. Therefore, CIPHER performed the best in terms of HMA detection accuracy among the frameworks. This inference is further reinforced by the fact that CIPHER had the best TPR, TNR, FPR, and FNR among the frameworks. However, in terms of communication overheads, CONTRAST had the best values, followed by PANOP and CIPHER. In execution overhead, CONTRAST had the best values, followed by WINDRAIN and CIPHER. Since CONTRAST, PANOP, and WINDRAIN had far lower HMA detection accuracy than CIPHER, and PANOP had zero energy savings, CIPHER did the overall best in these metrics as well. During scarce attack conditions, CONTRAST had the best values across all metrics. CIPHER had marginally lower second-best values and the best execution overhead value. Thus, CIPHER performed comparably with comparable state-of-the-art frameworks.

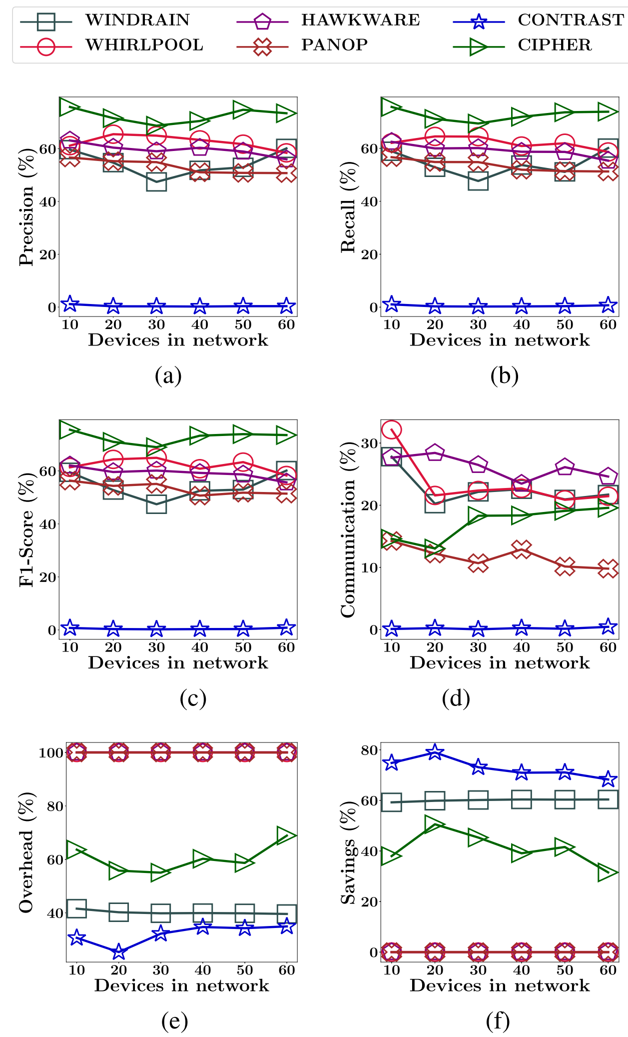


Fig. 5. Performance of the proposed framework during frequent attacks presented as percentages of the maximum possible value in their category. (a) Precision. (b) Recall. (c) $F1$-score. (d) Peer communication. (e) Execution overhead. (f) Executions avoided.

Fig. 4(a) presents the accuracy of the frameworks during frequent attacks for different network sizes. It shows that CIPHER had the highest HMA detection accuracy among all the frameworks. Fig. 4(b) presents the average percentage of nodes running IDS during frequent attack conditions for varying network sizes. It is seen that WINDRAIN used the fewest devices for placement, followed by CIPHER. This is because of the random-based placement scheme of WINDRAIN and the nonconservative placement scheme of CIPHER. All other frameworks used all available devices in the network for IDS placement. Similarly, Fig. 4(c) presents the accuracy of the frameworks during scarce attacks for different network sizes. It is seen that CIPHER maintained comparable HMA detection accuracy with that of the state-of-the-art frameworks.

TABLE IV
COMPARISON OF EXPERIMENTAL RESULTS DURING FREQUENT AND SCARCE ATTACK CONDITIONS PRESENTED AS PERCENTAGES (%)

Metric	Frequent attack conditions						Scarce attack conditions					
	WR [19]	**WP [26]**	**HW [9]**	**PP [10]**	**CT [7]**	**CIPHER**	**WR [19]**	**WP [26]**	**HW [9]**	**PP [10]**	**CT [7]**	**CIPHER**
ACC	83.09	78.89	74.79	69.62	64.66	**87.13**	**99.87**	99.61	81.99	93.72	**99.87**	99.83
PRC	54.81	61.32	58.65	53.01	1.25	**71.44**	**100.0**	99.64	77.28	93.06	**100.0**	99.95
REC	54.04	62.06	59.2	53.47	0.48	**72.58**	99.81	99.81	98.49	98.8	**99.82**	99.81
F1S	54.21	62.18	59.23	53.28	0.46	**72.69**	**99.91**	99.73	86.44	95.6	**99.91**	99.88
TPR	54.04	62.06	59.2	53.47	0.48	**72.58**	99.81	99.81	98.49	98.8	**99.82**	99.81
TNR	78.55	69.38	61.68	54.46	64.34	**81.54**	**100.0**	99.74	81.96	93.85	**100.0**	99.96
FPR	21.45	30.62	38.32	45.54	35.66	**18.46**	**0.00**	0.26	18.04	6.15	**0.00**	0.04
FNR	45.96	37.94	40.8	46.53	99.52	**27.42**	0.19	0.19	1.51	1.20	**0.18**	0.19
CO	22.61	23.58	26.12	11.78	**0.29**	17.24	9.66	11.50	36.17	11.25	**0.18**	8.03
EO	39.95	100.0	100.0	100.0	**27.13**	59.08	39.97	100.0	100.0	100.0	35.32	**26.29**

CO = Communication Overhead, EO = Execution Overhead, WR = WINDRAIN, WP = WHIRLPOOL, HW = HAWKWARE, PP = PANOP, CT = CONTRAST.

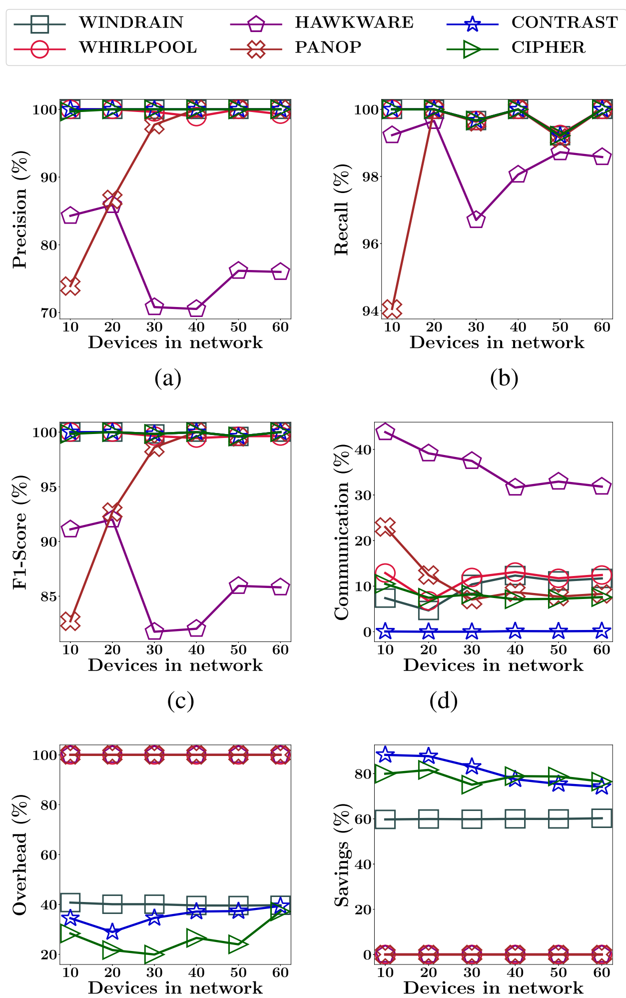


Fig. 6. Performance of the proposed framework during scarce attacks presented as percentages of the maximum possible value in their category. (a) Precision. (b) Recall. (c) $F1$-score. (d) Peer communication. (e) Execution overhead. (f) Executions avoided.

Fig. 4(d) presents the average percentage of nodes running IDS during scarce attack conditions for varying network sizes. A distribution similar to Fig. 4(b) can be seen here as well. Therefore, CIPHER achieved a high HMA detection accuracy without using all available devices for IDS placement.

Fig. 5 presents the performance of the frameworks under frequent attack conditions. Fig. 5(a) shows that CIPHER had the highest precision score among all frameworks, followed by WHIRLPOOL. Fig. 5(b) shows that CIPHER had the highest recall value, followed by WHIRLPOOL. Further, Fig. 5(c) shows that CIPHER had the highest F1-score, followed by HAWKWARE. Therefore, CIPHER portrayed the best overall performance in terms of HMA detection. Fig. 5(d) shows that CONTRAST incurred the lowest peer communication overhead, while HAWKWARE incurred the highest. CIPHER remained comparable with the other frameworks. This is because CONTRAST collaborated with peer devices less often, resulting in the lowest HMA detection accuracy. Fig. 5(e) shows that CONTRAST had the lowest execution overhead, followed by WINDRAIN, while WHIRLPOOL, HAWKWARE, and PANOP had the maximal overhead due to their IDSs executing at all times. CIPHER incurred intermediate execution overhead because it prioritized HMA detection over energy conservation during frequent attack conditions. Fig. 5(f) reinforces the above statement with the highest IDS execution savings done by CONTRAST, followed by WINDRAIN due to their low execution overhead, while CIPHER came in at third. Therefore, during frequent attack conditions, CIPHER showed the best HMA detection prowess and energy savings while performing comparably in peer communication and execution overheads to other equivalently accurate frameworks.

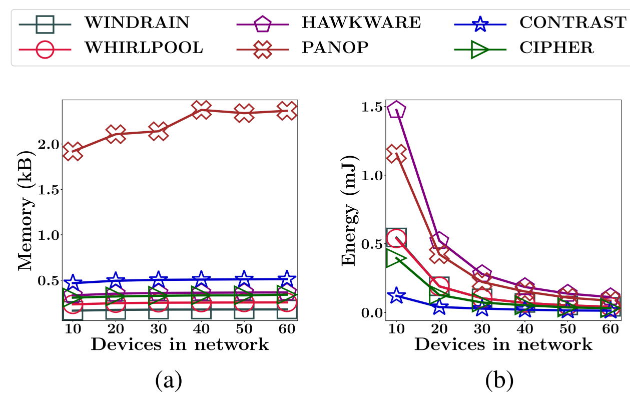


Fig. 7. Memory and energy consumption of the frameworks during attacks. (a) Memory consumption. (b) Energy consumption.

Fig. 6 presents the performance of the frameworks under scarce attack conditions. Fig. 6(a) shows that all frameworks, other than HAWKWARE, exhibited high precision values for most of the network sizes, with CIPHER having the overall

TABLE V
COMPARISON OF MODEL SIZES AND EXECUTION TIMES PER CYCLE

Metric	WR [19]	WP [26]	HW [9]	PP [10]	CT [7]	CIPHER
Size (kB)	1.79	1.62	3.42	1.28	0.08	2.07
Time (s)	0.03	0.09	0.23	0.18	0.03	0.10

second-best value. Fig. 6(b) shows that all frameworks, other than HAWKWARE, exhibited similar recall values for most of the network sizes. Similarly, Fig. 6(c) shows that all frameworks, other than HAWKWARE, exhibited similar $F1$-scores for most of the network sizes. Therefore, all frameworks, other than HAWKWARE, performed comparably in HMA detection accuracy during scarce attack conditions. Fig. 6(d) shows that all frameworks, other than HAWKWARE and CONTRAST, incurred similar peer communication overheads. While HAWKWARE incurred the highest communication overhead, CONTRAST incurred the lowest. Fig. 6(e) shows that CONTRAST and CIPHER had the lowest execution overheads, followed by WINDRAIN, while WHIRLPOOL, HAWKWARE, and PANOP had the maximal overhead due to their IDSs executing at all times. Fig. 6(f) shows that CONTRAST and CIPHER saved the most IDS executions during scarce attack conditions, followed by WINDRAIN, while WHIRLPOOL, HAWKWARE, and PANOP had the lowest IDS execution savings due to their constant IDS executions.

Fig. 7 shows the memory and energy consumptions of the frameworks during attacks. Fig. 7(a) shows that PANOP consumed the maximum memory while the other frameworks remained comparable. This is because the host devices implementing PANOP communicate their features to the central detection system. Similarly, Fig. 7(b) shows that HAWKWARE consumed the maximum energy, closely followed by PANOP, while the other frameworks remained comparable. This is because HAWKWARE implements both its network and host behavior-based detectors in the host devices. Furthermore, PANOP incurs communication overhead, in addition to that for collaboration, due to its centralized scheme. Table V shows the model sizes of the frameworks and their execution times per cycle. HAWKWARE had the largest model size, followed by CIPHER, while CONTRAST had the smallest. Even with a larger model size, CIPHER had a very low memory footprint [Fig. 7(a)], due to its subschemes. Also, HAWKWARE had the longest execution time per cycle, followed by PANOP, while CONTRAST had the shortest. CIPHER remained comparable with frameworks having similar accuracy. Therefore, CIPHER maintained high HMA detection performance with comparable communication and execution overheads, and saved more energy than comparable state-of-the-art frameworks.

D. Inference and Discussion

The above results reinforce that during frequent attack conditions, CIPHER achieves the highest HMA detection accuracy while maintaining comparable peer communication overheads, execution overhead, and energy savings to the state-of-the-art frameworks with similar HMA detection prowess. Additionally, during scarce attack conditions, CIPHER saves the maximum possible energy while maintaining comparable HMA detection accuracy, communication overhead, and execution overhead to the state-of-the-art frameworks. This confirms that CIPHER is able to prioritize between its objectives as per the state of the device, such that it can emphasize either energy-saving during scarce attack conditions, or HMA detection accuracy during frequent attack conditions, while it maintains the other metrics comparable to the state-of-the-art.

Edge conditions, such as dynamic parent changes, sparse networks, and uneven degree distributions, pose challenges to IDS placement coverage. Mobile nodes can dynamically change their parents, altering the DODAG structure. However, a child node that changes its parent selects another parent in its communication range. Since placement nodes are selected based on the same, this will not affect coverage. For a large topology with hundreds of nodes, the communication overhead in distributed collaboration is nontrivial. In this situation, the nodes' collaboration radius is adjusted to balance accuracy and communication overhead. A similar adjustment to the collaboration radius improves accuracy in sparse networks. Additionally, sparse networks have nodes with fewer neighbors in their communication ranges. However, since internal nodes are selected for placement, the IDS still provides multihop network coverage. In networks with uneven neighbor degree distributions, dense areas are trivially covered, and sparse areas are covered via multihop paths, ensuring total coverage. In these networks, each node can adjust its collaboration radius to balance accuracy and communication overhead.

Game-theoretic placement schemes, alongside the randomized scheme, are identified as a future augmentation to enhance the placement scheme's robustness. Also, as CIPHER does not use a trust system, Byzantine behavior, compromised collaborating nodes, and noisy collaboration responses will affect its performance. In the future, a reputation-based trust system will be augmented to ensure robust response aggregation and immunity against collusion [38]. Additionally, CIPHER does not have robust immunity against outliers. In the future, outlier rejection will also be augmented to the proposed framework. Furthermore, federated learning (FL) enables collaborators to have IDS detectors tailored to their data, optimizing host resource usage. In the future, FL will be used to customize the IDS models to their target data. Moreover, blockchains provide robust integrity checking mechanisms to enforce honesty in collaboration. This is declared as another future scope. Lastly, this work covers IDS deployment in homogeneous networks. In the future, highly heterogeneous networks will be considered for IDS deployment.

V. CONCLUSION

The IoT networks require security mechanisms to defend against attacks. Advanced attacks like HMAs mimic legitimate behavior to evade detection and leave a smaller attack footprint. Therefore, hybrid IDSs can detect HMAs better due to the network context availability during device behavior analysis. CIDSs improve on the detection accuracy further by using peer consultation. These CIDSs execute constantly throughout the deployment phase to have their detection results ready for collaboration requests. However, such an approach consumes resources of the host devices. Therefore, this work proposes a CIPHER for IoT networks. CIPHER uses a hybrid CIDS for collaborative HMA detection, a communication range-based placement scheme to ensure seamless coverage, a request-response and feature-monitor-based scheduling scheme to ensure resource-saving, and a device state history-based objective selection scheme to prioritize between detection accuracy and energy savings. Extensive testbed experimentation has proved that CIPHER achieves 87.13% HMA detection accuracy with 39.62% fewer IDS executions, while maintaining comparable performance to the state-of-the-art in peer communication and execution overheads during frequent attacks. Additionally, CIPHER saves 78.46% IDS executions with 99.83% HMA detection accuracy and comparable communication and execution overheads to the state-of-the-art frameworks during scarce attack conditions.

References

[1] T. Wang et al., "AudioGuard: Omnidirectional indoor intrusion detection using audio device," *ACM Trans. Internet Things*, vol. 5, no. 1, pp. 1–22, Feb. 2024.

[2] C. Liu, B. Chen, W. Shao, C. Zhang, K. K. L. Wong, and Y. Zhang, "Unraveling attacks to machine-learning-based IoT systems: A survey and the open libraries behind them," *IEEE Internet Things J.*, vol. 11, no. 11, pp. 19232–19255, Jun. 2024.

[3] B. D. Son et al., "Adversarial attacks and defenses in 6G network-assisted IoT systems," *IEEE Internet Things J.*, vol. 11, no. 11, pp. 19168–19187, Jun. 2024.

[4] C. Parampalli, R. Sekar, and R. Johnson, "A practical mimicry attack against powerful system-call monitors," in *Proc. ACM Symp. Inf., Comput. Commun. Secur.*, Mar. 2008, pp. 156–167.

[5] R. W. Anwer, M. Abrar, A. Salam, and F. Ullah, "TEAD: Trust-enhanced anomaly detection framework for intrusion detection in IoT-enabled wireless sensor networks (WSNs)," *Wireless Netw.*, vol. 31, no. 6, pp. 4179–4197, Aug. 2025.

[6] T. E. T. Djaidja, B. Brik, S. Mohammed Senouci, A. Boualouache, and Y. Ghamri-Doudane, "Early network intrusion detection enabled by attention mechanisms and RNNs," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 7783–7793, 2024.

[7] S. Sanyal, M. Khatua, and P. Chattopadhyay, "An adaptive IDS scheduling framework for host based mimicry attack detection in IoT networks," in *Proc. IEEE Int. Conf. Adv. Netw. Telecommun. Syst. (ANTS)*, Dec. 2024, pp. 1–6.

[8] J. He, C. Tang, W. Li, T. Li, L. Chen, and X. Lan, "BR-HIDF: An anti-sparsity and effective host intrusion detection framework based on multi-granularity feature extraction," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 485–499, 2024.

[9] S. Ahn, H. Yi, Y. Lee, W. R. Ha, G. Kim, and Y. Paek, "Hawkware: Network intrusion detection based on behavior analysis with ANNs on an IoT device," in *Proc. 57th ACM/IEEE Design Autom. Conf. (DAC)*, Jul. 2020, pp. 1–6.

[10] H. Kim et al., "Panop: Mimicry-resistant ANN-based distributed NIDS for IoT networks," *IEEE Access*, vol. 9, pp. 111853–111864, 2021.

[11] F. Erlacher and F. Dressler, "On high-speed flow-based intrusion detection using snort-compatible signatures," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 1, pp. 495–506, Jan. 2022.

[12] K. Agrawal, T. Alladi, A. Agrawal, V. Chamola, and A. Benslimane, "NovelADS: A novel anomaly detection system for intra-vehicular networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 11, pp. 22596–22606, Nov. 2022.

[13] H. Babbar and S. Rani, "FRHIDS: Federated learning recommender hybrid intrusion detection system model in software-defined networking for consumer devices," *IEEE Trans. Consum. Electron.*, vol. 70, no. 1, pp. 2492–2499, Feb. 2024.

[14] A. Uprety, D. B. Rawat, and B. Sadler, "Human immune system inspired security for federated learning-empowered Internet of Things," *ACM Trans. Internet Things*, vol. 6, no. 2, pp. 1–18, May 2025.

[15] U. Sabeel, S. S. Heydari, K. El-Khatib, and K. Elgazzar, "Unknown, atypical and polymorphic network intrusion detection: A systematic survey," *IEEE Trans. Netw. Service Manage.*, vol. 21, no. 1, pp. 1190–1212, Jul. 2023.

[16] M. Huang et al., "ARIoTEDef: Adversarially robust IoT early defense system based on self-evolution against multi-step attacks," *ACM Trans. Internet Things*, vol. 5, no. 3, pp. 1–34, Aug. 2024.

[17] H. Kayan, R. Heartfield, O. Rana, P. Burnap, and C. Perera, "CASPER: Context-aware IoT anomaly detection system for industrial robotic arms," *ACM Trans. Internet Things*, vol. 5, no. 3, pp. 1–36, Aug. 2024.

[18] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2018, pp. 1–15.

[19] S. P. Chung and A. K. Mok, "On random-inspection-based intrusion detection," in *Recent Advances in Intrusion Detection (RAID)*. Cham, Switzerland: Springer, 2006, pp. 165–184.

[20] A. A. Wardana and P. Sukarno, "Taxonomy and survey of collaborative intrusion detection system using federated learning," *ACM Comput. Surveys*, vol. 57, no. 4, pp. 1–36, Apr. 2025.

[21] R. Xing, Z. Su, and Y. Wang, "Collaborative intrusion detection approach based on blockchain in Internet of Vehicles," *IEEE Internet Things J.*, vol. 12, no. 9, pp. 11965–11976, May 2025.

[22] H. Jalil Hadi, Y. Cao, S. Li, Y. Hu, J. Wang, and S. Wang, "Real-time collaborative intrusion detection system in UAV networks using deep learning," *IEEE Internet Things J.*, vol. 11, no. 20, pp. 33371–33391, Oct. 2024.

[23] Y. Meidan, D. Avraham, H. Libhaber, and A. Shabtai, "CADeSH: Collaborative anomaly detection for smart homes," *IEEE Internet Things J.*, vol. 10, no. 10, pp. 8514–8532, May 2023.

[24] P. Kannadiga and M. Zulkernine, "DIDMA: A distributed intrusion detection system using mobile agents," in *Proc. 6th Int. Conf. Softw. Eng., Artif. Intell., Netw. Parallel/Distributed Comput. 1st ACIS Int. Workshop Self-Assembling Wireless Netw.*, 2005, pp. 238–245.

[25] W. Li, W. Meng, J. Parra-Arnau, and K. R. Choo, "Enhancing challenge-based collaborative intrusion detection against insider attacks using spatial correlation," in *Proc. IEEE Conf. Dependable Secure Comput. (DSC)*, Jan. 2021, pp. 1–8.

[26] M. E. Locasto, J. J. Parekh, S. Stolfo, A. D. Keromytis, T. G. Malkin, and V. Misra, "Collaborative distributed intrusion detection," Dept. Computer Science Technical Reports, Columbia Univ., Tech. Rep. CUCS-012-04, 2004.

[27] J. E. Tapiador and J. A. Clark, "Masquerade mimicry attack detection: A randomised approach," *Comput. Secur.*, vol. 30, no. 5, pp. 297–310, Jul. 2011.

[28] P. Thubert, *An Architecture for IPv6 Over the Time-slotted Channel Hopping Mode of IEEE 802.15.4 (6TiSCH)*, document RFC 9030, 2021.

[29] R. Karwayun, P. Sharma, M. Sainger, N. Joshi, and S. Manna, "Role of spyware in the intelligent digital age: A comparative analysis," in *Proc. 4th Int. Conf. Advancement Electron. Commun. Eng. (AECE)*, Nov. 2024, pp. 1058–1066.

[30] Y. Xu, D. Li, Q. Li, and S. Xu, "Malware evasion attacks against IoT and other devices: An empirical study," *Tsinghua Sci. Technol.*, vol. 29, no. 1, pp. 127–142, Feb. 2024.

[31] D. Wagner and P. Soto, "Mimicry attacks on host-based intrusion detection systems," in *Proc. 9th ACM Conf. Comput. Commun. Secur.*, Nov. 2002, pp. 255–264.

[32] R. DiPietro and G. D. Hager, "Deep learning: RNNs and LSTM," in *Handbook of Medical Image Computing and Computer Assisted Intervention*, 2020, pp. 503–519.

[33] W. H. L. Pinaya, S. Vieira, R. Garcia-Dias, and A. Mechelli, "Autoencoders," in *Machine Learning*. Amsterdam, The Netherlands: Elsevier, 2020, pp. 193–208.

[34] M. Riedmiller and A. Lernen, "Multi layer perceptron," Univ. Freiburg, Tech. Rep. 24, 2014, vol. 24.

[35] J. T. Hancock and T. M. Khoshgoftaar, "Survey on categorical data for neural networks," *J. Big Data*, vol. 7, no. 1, p. 28, Dec. 2020.

[36] G. Oikonomou, S. Duquennoy, A. Elsts, J. Eriksson, Y. Tanaka, and N. Tsiftes, "The contiki-NG open source operating system for next generation IoT devices," *SoftwareX*, vol. 18, pp. 101089–101097, Jun. 2022.

[37] C. Adjih et al., "FIT IoT-LAB: A large scale open experimental IoT testbed," in *Proc. IEEE 2nd World Forum Internet Things (WF-IoT)*, Dec. 2015, pp. 459–464.

[38] F. Ullah et al., "Deep trust: A novel framework for dynamic trust and reputation management in the Internet of Things (IoT)-based networks," *IEEE Access*, vol. 12, pp. 87407–87419, 2024.